Coding For Placement Series

Problem Solving Bootcamp Track

CDP 221 to 232

STUDENT HANDOUT

BIZOTIC

Empowering students towards their first job!

v1

TABLE OF CONTENTS

#	Block	Page
1	**CDP 221 - Reactivation and Scenario Decoder Framework**	**05**
	Program 1 - Calculator Utility	
	Program 2 - Student Grade Classifier	
	Program 3 - Attendance Eligibility Checker	
	Program 4 - Voting Poll Summarizer	
	Program 5 - Number Pyramid Printer	
	Challenge 1 - Sum of Digits	
	Challenge 2 - Odd/Even Count in Array	
	Challenge 3 - Max and Min in Array	
2	**CDP 222 - Conditional Logic Mastery**	**09**
	Program 1 - Age Category Classifier	
	Program 2 - Electricity Bill Calculator	
	Program 3 - Income Tax Calculator	
	Program 4 - Insurance Premium Calculator	
	Program 5 - Movie Ticket Price Calculator	
	Challenge 1 - Leap Year	
	Challenge 2 - Shipping Speed Selector	
	Challenge 3 - BMI Calculator	
3	**CDP 223 - Loop Patterns and Iteration**	**13**
	Program 1 - Factorial and Digit Sum	
	Program 2 - Prime Number Checker	
	Program 3 - Multiplication Table Printer	
	Program 4 - Sequence Generator	
	Program 5 - Star Pattern Diamond	
	Challenge 1 - Sum of Natural Numbers	
	Challenge 2 - Reverse a Number	
	Challenge 3 - GCD of Two Numbers	
4	**CDP 224 - Array Traversal Patterns**	**17**
	Program 1 - Sales Tracker	
	Program 2 - Max Consecutive Ones	
	Program 3 - Range Sum Query	

#	Block	Page
	Program 4 - Minimum Value to Get Positive Step by Step Sum	
	Program 5 - Unique Number of Occurrences	
	Challenge 1 - LC 1512 Number of Good Pairs	
	Challenge 2 - LC 724 Find Pivot Index	
	Challenge 3 - LC 287 Find the Duplicate Number	
5	**CDP 225 - Two-Pointer and Window Patterns**	**21**
	Program 1 - Pair Sum for Target	
	Program 2 - Continuous Fitness Streak Monitor	
	Program 3 - Squares of a Sorted Array	
	Program 4 - Ride Sharing Load Balancer	
	Program 5 - Defuse the Bomb	
	Challenge 1 - LC 1346 Check If N and Its Double Exist	
	Challenge 2 - LC 167 Two Sum II Sorted	
	Challenge 3 - LC 643 Maximum Average Subarray I	
6	**CDP 226 - String Processing Patterns**	**25**
	Program 1 - Password Strength Checker	
	Program 2 - Student Name Anagram Matcher	
	Program 3 - Substrings of Size Three with Distinct Characters	
	Program 4 - Address Parser and Validator	
	Program 5 - Add Strings	
	Challenge 1 - LC 942 DI String Match	
	Challenge 2 - LC 541 Reverse String II	
	Challenge 3 - LC 1456 Maximum Number of Vowels in a Substring of Given Length	
7	**CDP 227 - Data Structure Intuition**	**28**
	Program 1 - Undo Feature in Text Editor	
	Program 2 - Duplicate Detector in Customer Database	
	Program 3 - Baseball Game	
	Program 4 - Hospital Token Management System	
	Program 5 - Final Prices With a Special Discount in a Shop	
	Challenge 1 - Count the Number of Consistent Strings	
	Challenge 2 - Maximum Nesting Depth of the Parentheses	
	Challenge 3 - Uncommon Words from Two Sentences	

#	Block	Page
8	**CDP 228 - Search and Sort Foundations**	32
	Program 1 - Phone Directory Search	
	Program 2 - Binary Search	
	Program 3 - Bus Seat Availability Finder	
	Program 4 - Event Scheduler Conflict-Free	
	Program 5 - Third Maximum Number	
	Challenge 1 - Count Negative Numbers in a Sorted Matrix	
	Challenge 2 - Sqrt(x)	
	Challenge 3 - Peak Index in a Mountain Array	
9	**CDP 229 - Company Simulation Block 1 (Accenture-Style)**	36
	Problem 1 - Attendance Summary Report	
	Problem 2 - Salary Slip Generator	
	Problem 3 - Library Fine Calculator	
10	**CDP 230 - Company Simulation Block 2 (Visa-Style)**	39
	Problem 1 - Sliding Window Maximum Engagement	
	Problem 2 - Inventory Duplicate Auditor	
	Problem 3 - Campus Event Popularity Ranker	
11	**CDP 231 - Company Simulation Block 3 (Mixed-Pool)**	42
	Problem 1 - Multi-Category Discount Voucher System	
	Problem 2 - Clustered Task Assignment	
	Problem 3 - Subscription Revenue Analytics	
12	**CDP 232 - Pattern Synthesis and Placement Playbook**	45
	Program 1 - Most Active Reviewer	
	Program 2 - Smart Thermostat Controller	
	Program 3 - Corporate Flight Bookings	
	Program 4 - Top-Selling Products Dashboard	
	Program 5 - Max Consecutive Ones III	
	Challenge 1 - Array Partition	
	Challenge 2 - Find All Anagrams in a String	
	Challenge 3 - Flipping an Image	

CDP 221 - Reactivation and Scenario Decoder Framework

Program 1: Calculator Utility (Easy, 10 min)

Problem Statement:

Read two integers `a` and `b` and an operator character (`+`, `-`, `*`, `/`). Print the result. For division, print integer division result (truncated). If `b == 0` and operator is `/`, print `ERROR: DIVIDE BY ZERO`.

Test Cases:

#	Input	Expected Output
1	10 3 +	13
2	10 3 -	7
3	10 3 *	30
4	10 3 /	3
5	10 0 /	ERROR: DIVIDE BY ZERO

Program 2: Student Grade Classifier (Medium, 25 min)

Problem Statement:

A college grading system assigns letter grades based on marks (out of 100):

- `A+`: 90 <= marks <= 100
- `A`: 80 <= marks < 90
- `B`: 70 <= marks < 80
- `C`: 60 <= marks < 70
- `D`: 50 <= marks < 60
- `F`: marks < 50
- Marks > 100 or < 0: `INVALID`

For N students (each with a name and marks), print `NAME: MARKS GRADE` or `NAME: MARKS INVALID`.

Test Cases:

#	Input	Expected Output
1	3: Alice 95, Bob 72, Carol 45	Alice A+, Bob B, Carol F
2	2: A 90, B 89	A A+, B A (boundary check)
3	2: X 101, Y -5	X INVALID, Y INVALID
4	1: Sam 50	Sam D (boundary)
5	1: Eve 100	Eve A+ (top boundary)

Program 3: Attendance Eligibility Checker (Medium, 20 min)

Problem Statement:

A college requires 75% attendance for exam eligibility. For N students, each given as (name, classes_attended, total_classes), print `NAME: ELIGIBLE` or `NAME: NOT-ELIGIBLE`.

Percentage = classes_attended / total_classes * 100. Eligibility: percentage >= 75.00.

Test Cases:

#	Input	Expected Output
1	3: Alice 40 50, Bob 30 50, Carol 38 50	Alice ELIGIBLE, Bob NOT, Carol ELIGIBLE
2	2: A 75 100, B 74 100	A ELIGIBLE (boundary), B NOT
3	1: Eve 45 45	Eve ELIGIBLE (100%)
4	1: Sam 0 50	Sam NOT-ELIGIBLE
5	1: Fay 37 50	Fay NOT-ELIGIBLE (74%)

Program 4: Voting Poll Summarizer (Hard, 25 min)

Problem Statement:

A class election has 5 candidates (numbered 1-5). N votes are cast (each a single integer 1-5). Invalid votes (outside 1-5) are ignored. Print:

- Vote count for each candidate (lines 1-5), one per line: `CANDIDATE <n>: <count>`.
- The winner: `WINNER: CANDIDATE <n>` (highest count; on ties, smallest candidate number).
- Total valid votes: `TOTAL VALID: <count>`.
- Total invalid votes: `INVALID: <count>`.

Test Cases:

#	Input (N, votes)	Winner
1	10: [1 2 3 1 2 1 4 5 1 2]	Winner 1 (4 votes)
2	5: [6 7 1 0 9]	All invalid except 1. Winner 1 (1). Invalid 4.
3	5: [3 3 3 3 3]	Winner 3 (5).
4	6: [1 2 1 2 1 2]	Winner 1 (tie, smallest id).
5	0 votes	Winner: NO VOTES

Program 5: Number Pyramid Printer (Hard, Stretch, 10 min)

Problem Statement:

Given N, print a number pyramid of N rows. Row i (1-indexed) contains i numbers, starting at 1 and incrementing. Example for N = 4: `1 2 3 4 5 6 7 8 9 10`

Test Cases:

#	N	Expected Output
1	4	4 rows as above
2	1	1
3	3	1 / 2 3 / 4 5 6
4	5	5 rows ending with 11 12 13 14 15
5	0	(no output)

Challenge 1: Sum of Digits (Bizotic Easy)

Problem Statement:

Read a non-negative integer N. Print the sum of its digits.

Test Cases:

#	Input	Expected Output
1	123	6
2	9	9
3	0	0
4	99999	45
5	1000000007	8

Challenge 2: Odd/Even Count in Array (Bizotic Easy)

Problem Statement:

Read N and N integers. Print `ODD: x, EVEN: y`.

Test Cases:

#	Input	Expected Output
1	5 1 2 3 4 5	ODD: 3, EVEN: 2
2	4 2 4 6 8	ODD: 0, EVEN: 4
3	3 1 3 5	ODD: 3, EVEN: 0
4	1 0	ODD: 0, EVEN: 1
5	6 -3 -2 0 0 7 -8	ODD: 2, EVEN: 4

Challenge 3: Max and Min in Array (Bizotic Easy)

Problem Statement:

Read N and N integers. Print `MAX: x, MIN: y`.

Test Cases:

#	Input	Expected Output
1	5 3 1 4 1 5	MAX: 5, MIN: 1
2	1 42	MAX: 42, MIN: 42
3	4 -5 -1 -10 -3	MAX: -1, MIN: -10
4	0	EMPTY
5	5 7 7 7 7 7	MAX: 7, MIN: 7

CDP 222 - Conditional Logic Mastery

Program 1: Age Category Classifier (Easy, 10 min)

Problem Statement:

Given a person's age (integer), classify them:

- 0-12: CHILD
- 13-19: TEEN
- 20-59: ADULT
- 60+: SENIOR
- Negative or above 150: INVALID

Print only the category label.

Test Cases:

#	Input	Output
1	10	CHILD
2	17	TEEN
3	45	ADULT
4	70	SENIOR
5	200	INVALID

Program 2: Electricity Bill Calculator (Medium, 25 min)

Problem Statement:

Electricity bill slabs (per unit consumed):

- Units 1-100: Rs 2 per unit
- Units 101-300: Rs 4 per unit
- Units 301-500: Rs 6 per unit
- Units 501+: Rs 10 per unit

Given units consumed, compute the bill slab-by-slab. Print `BILL: <amount>`.

Test Cases:

#	Units	Expected Output
1	50	BILL: 100
2	150	BILL: 400
3	300	BILL: 1000
4	500	BILL: 2200
5	600	BILL: 3200

Program 3: Income Tax Calculator (Medium, 20 min)

Problem Statement:

Annual income tax slabs:

- Up to Rs 2,50,000: 0% (no tax)
- Rs 2,50,001 to Rs 5,00,000: 5% on the portion above 2.5L
- Rs 5,00,001 to Rs 10,00,000: flat Rs 12,500 (from the 2.5L-5L slab) + 20% on portion above 5L
- Above Rs 10,00,000: flat Rs 12,500 + Rs 1,00,000 (from the 5L-10L slab) + 30% on portion above 10L

Print `TAX: <amount>`.

Test Cases:

#	Income	Expected Tax
1	200000	TAX: 0
2	400000	TAX: 7500 (150000 * 0.05)
3	800000	TAX: 72500 (12500 + 300000*0.20)
4	1500000	TAX: 262500 (12500 + 100000 + 500000*0.30)
5	250000	TAX: 0 (boundary)

Program 4: Insurance Premium Calculator (Hard, 25 min)

Problem Statement:

Compute health insurance monthly premium:

1. Base premium = Rs 1000.
2. Age surcharge: if age > 50, add 50% of base. If age > 65, add another 50% of base (total 100% surcharge).
3. Gender factor: if FEMALE, multiply current premium by 0.95 (5% discount).
4. Loyalty discount: if years_as_customer >= 5, subtract 10% of CURRENT premium (after above adjustments).
5. Round final to 2 decimals.

Inputs: age (int), gender (MALE / FEMALE), years_as_customer (int). Print `PREMIUM: <amount>`.

Test Cases:

#	Age, Gender, Years	Expected Output
1	30, MALE, 2	1000.00 (no modifiers)
2	55, MALE, 3	1500.00 (age surcharge only)
3	70, FEMALE, 10	2000 + 100% = 2000; female 0.95 = 1900; loyalty -10% = 1710.00
4	30, FEMALE, 5	1000 * 0.95 * 0.90 = 855.00

#	Age, Gender, Years	Expected Output
5	45, MALE, 5	1000 * 0.90 = 900.00

Program 5: Movie Ticket Price Calculator (Hard, Stretch, 15 min)

Problem Statement:

Movie ticket price depends on:

- Day: `WEEKDAY` (Mon-Thu) base 200, `WEEKEND` (Fri-Sun) base 300, `HOLIDAY` base 400.
- Show time: `MORNING` -20%, `MATINEE` base, `EVENING` +10%, `NIGHT` +20%.
- Seat category: `NORMAL` base, `PREMIUM` +50% on current, `VIP` +100% on current.

Inputs: day, time, seat. Apply multipliers in order: day base, then time modifier, then seat modifier. Print `PRICE: <amount>`.

Test Cases:

#	Inputs	Expected
1	WEEKDAY MATINEE NORMAL	200.00
2	WEEKEND EVENING PREMIUM	300*1.1*1.5 = 495.00
3	HOLIDAY NIGHT VIP	400*1.2*2.0 = 960.00
4	WEEKDAY MORNING NORMAL	200*0.8 = 160.00
5	WEEKEND MATINEE NORMAL	300.00

Challenge 1: Leap Year (Bizotic Easy)

Problem Statement:

Read a single integer year. Determine whether the year is a leap year using the standard Gregorian rule: a year is leap if it is divisible by 4, EXCEPT it is NOT leap if it is divisible by 100, EXCEPT it IS leap again if it is divisible by 400. Print `YES` if the year is leap, otherwise print `NO`. Inputs are positive integers in the range [1, 9999].

Test Cases:

#	Input	Expected Output
1	2024	YES
2	2023	NO
3	2000	YES
4	1900	NO
5	2400	YES

Challenge 2: Shipping Speed Selector (Bizotic Easy)

Problem Statement:

A shipping carrier selects a delivery mode based on package weight (kilograms) and a priority flag (0 or 1). Read the weight (a real number) and the priority flag (an integer). Apply the rules in order: if the priority flag is 1 AND weight is at most 5.0 kg, ship by `AIR`; otherwise if weight is at most 20.0 kg, ship by `ROAD`; otherwise ship by `SEA`. Print only the shipping mode label. Note: a non-priority package weighing 5 kg or less still ships by ROAD, not AIR — the priority flag is required for AIR.

Test Cases:

#	Input	Expected Output
1	3.5 1	AIR
2	5.0 1	AIR
3	3.5 0	ROAD
4	20.0 0	ROAD
5	25.0 1	SEA

Challenge 3: BMI Calculator (Bizotic Easy)

Problem Statement:

Read weight (kilograms, real number) and height (meters, real number). Compute Body Mass Index as `BMI = weight / (height * height)`. Print the BMI rounded to 2 decimals followed by a space and the category label, where categories are: BMI below 18.5 → `UNDERWEIGHT`; 18.5 to below 25.0 → `NORMAL`; 25.0 to below 30.0 → `OVERWEIGHT`; 30.0 and above → `OBESE`. If height is zero or negative, print `INVALID` and stop (no BMI value is printed in that case).

Test Cases:

#	Input	Expected Output
1	60 1.7	20.76 NORMAL
2	45 1.7	15.57 UNDERWEIGHT
3	80 1.7	27.68 OVERWEIGHT
4	100 1.7	34.60 OBESE
5	60 0	INVALID

CDP 223 - Loop Patterns and Iteration

Program 1: Factorial and Digit Sum (Easy, 15 min)

Problem Statement:

Read N. Compute N!. Print N!, followed by the sum of digits of N!.

Test Cases:

#	Input	Expected Output
1	5	120 3
2	10	3628800 27
3	0	1 1
4	1	1 1
5	-3	INVALID

Program 2: Prime Number Checker (Medium, 20 min)

Problem Statement:

Given N, print PRIME or NOT PRIME. Use the sqrt(N) optimization.

Test Cases:

#	Input	Expected Output
1	2	PRIME
2	7	PRIME
3	9	NOT PRIME
4	25	NOT PRIME
5	97	PRIME

Program 3: Multiplication Table Printer (Medium, 15 min)

Problem Statement:

Read N and M. Print the NxM multiplication table. Row i, column j prints i*j, tab-separated.

Test Cases:

#	Input	Expected Output
1	2 3	1 2 3 2 4 6
2	1 1	1
3	3 3	1 2 3 2 4 6 3 6 9
4	1 5	1 2 3 4 5

#	Input	Expected Output
5	5 1	1 2 3 4 5

Program 4: Sequence Generator (Hard, 30 min)

Problem Statement:

Given type (FIBONACCI, ARITHMETIC, GEOMETRIC), length N, and seed parameters. Generate and print the sequence.

Test Cases:

#	Input	Expected Output
1	FIBONACCI 0 1 7	0 1 1 2 3 5 8
2	ARITHMETIC 5 3 5	5 8 11 14 17
3	GEOMETRIC 2 3 5	2 6 18 54 162
4	FIBONACCI 1 1 1	1
5	ARITHMETIC 10 -2 4	10 8 6 4

Program 5: Star Pattern Diamond (Hard, Stretch, 15 min)

Problem Statement:

Print a diamond of height (2N-1). Upper half: rows 1..N with i stars centered. Lower half: rows N-1 down to 1.

Test Cases:

#	Input	Expected Output
1	1	*
2	2	* * *
3	3	* * * * *
4	4	* * * * * * *

#	Input	Expected Output
5	5	* * * * * * * * *

Challenge 1: Sum of Natural Numbers (Bizotic Easy)

Problem Statement:

Read a non-negative integer N. Print the sum 1+2+...+N (use the closed-form n*(n+1)/2). *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	1	1
2	5	15
3	100	5050
4	0	0
5	1000000	500000500000

Challenge 2: Reverse a Number (Bizotic Easy)

Problem Statement:

Read an integer N. Print the integer obtained by reversing its digits, preserving sign for negatives. *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	123	321
2	1000	1
3	7	7
4	0	0
5	-456	-654

Challenge 3: GCD of Two Numbers (Bizotic Easy, Euclidean)

Problem Statement:

Read two integers a and b. Print the greatest common divisor using the Euclidean algorithm.
(written by Claude as fallback; source had no Problem Statement)

Test Cases:

#	Input	Expected Output
1	12 18	6
2	17 13	1
3	100 0	100
4	21 21	21
5	48 36	12

CDP 224 - Array Traversal Patterns

Program 1: Sales Tracker (Easy, 15 min)

Problem Statement:

N daily sales figures. Print daily max, daily min, total, average (2 decimals), and count of days with sales > average.

Test Cases:

#	Input	Expected Output
1	5 100 200 150 300 250	MAX: 300 MIN: 100 TOTAL: 1000 AVG: 200.00 ABOVE AVG: 2
2	1 500	MAX: 500 MIN: 500 TOTAL: 500 AVG: 500.00 ABOVE AVG: 0
3	4 10 10 10 10	MAX: 10 MIN: 10 TOTAL: 40 AVG: 10.00 ABOVE AVG: 0
4	0	NO DATA
5	3 1 2 3	MAX: 3 MIN: 1 TOTAL: 6 AVG: 2.00 ABOVE AVG: 1

Program 2: Max Consecutive Ones (LeetCode 485, Medium, 20 min)

Problem Statement:

Binary array of N values (0 or 1). Find the maximum length of consecutive 1s.

Test Cases:

#	Input	Expected
1	6: [1 1 0 1 1 1]	3
2	6: [1 0 1 1 0 1]	2
3	4: [0 0 0 0]	0
4	1: [1]	1
5	5: [1 1 1 1 1]	5

Program 3: Range Sum Query (Medium, 20 min)

Problem Statement:

Array of N. Q queries, each (l, r) 0-indexed inclusive. Print sum of arr[l..r]. Build prefix sum O(N), query O(1).

Test Cases:

#	Input	Expected Output
1	5 1 2 3 4 5 3 0 4 1 3 2 2	15 9 3
2	1 100 1 0 0	100
3	4 -1 -2 -3 -4 2 0 1 2 3	-3 -7
4	3 5 5 5 1 0 2	15
5	5 10 20 30 40 50 4 0 0 0 1 1 4 4 4	10 30 140 50

Program 4: Minimum Value to Get Positive Step by Step Sum (LeetCode 1413, Hard, 25 min)

Problem Statement:

Given an array of integers (positive or negative), find the minimum positive starting value such that the running cumulative sum stays >= 1 at every step.

Test Cases:

#	Input	Expected
1	5: [-3 2 -3 4 2]	5
2	2: [1 2]	1
3	3: [1 -2 -3]	5
4	3: [0 0 0]	1
5	1: [-5]	6

Program 5: Unique Number of Occurrences (LeetCode 1207, Hard, Stretch, 15 min)

Problem Statement:

Given an array of integers, return true if all the values' frequencies are unique. (Each value appears some number of times; return true iff no two distinct values share the same frequency.)

Test Cases:

#	Input	Expected
1	6: [1 2 2 1 1 3]	true

#	Input	Expected
2	2: [1 2]	false
3	10: [-3 0 1 -3 1 1 1 -3 10 0]	true
4	1: [5]	true
5	3: [4 4 4]	true

Challenge 1: LC 1512 Number of Good Pairs

Problem Statement:

Read N then N integers. Count the pairs (i,j) where i<j and arr[i]==arr[j]. Print the count. *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	4 1 2 3 1	1
2	6 1 1 1 1 1 1	15
3	3 1 2 3	0
4	1 7	0
5	5 1 2 1 2 1	4

Challenge 2: LC 724 Find Pivot Index

Problem Statement:

Read N then N integers. Print the smallest index i where the sum of elements left of i equals the sum to the right of i, or -1 if no such index exists. *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	6 1 7 3 6 5 6	3
2	3 1 2 3	-1
3	1 0	0
4	6 -1 -1 -1 -1 -1 0	2
5	5 1 2 3 4 5	-1

Challenge 3: LC 287 Find the Duplicate Number

Problem Statement:

An array of N+1 integers, each in [1, N]. Exactly one value appears more than once (could be 2+ times). Find the duplicate.

Test Cases:

#	Input	Expected
1	5: [1 3 4 2 2]	2
2	5: [3 1 3 4 2]	3
3	2: [1 1]	1
4	3: [1 1 2]	1
5	5: [2 2 2 2 2]	2

CDP 225 - Two-Pointer and Window Patterns

Program 1: Pair Sum for Target (Easy, 15 min)

Problem Statement:

Sorted prices array of N items. Find 2 items summing to budget B. Print "PAIR FOUND: x AND y" or "NO PAIR FOUND".

Test Cases:

#	Input	Output
1	5,15,[2 4 7 11 15]	PAIR FOUND: 4 AND 11
2	4,100,[2 4 7 11]	NO PAIR FOUND
3	6,17,[2 3 5 8 10 15]	PAIR FOUND: 2 AND 15
4	1,5,[5]	NO PAIR FOUND
5	5,10,[3 3 5 5 7]	PAIR FOUND: 3 AND 7

Program 2: Continuous Fitness Streak Monitor (Medium, 25 min)

Problem Statement:

N step counts. Find longest contiguous streak where average >= T. Print "LONGEST STREAK: D DAYS / START DAY: s / END DAY: e" or "NO STREAK".

Test Cases:

#	Input	Output
1	7,5000,[6000 5500 4800 5200 5300 5100 4900]	7 DAYS, 1-7
2	5,10000,[2000 3000 4000 5000 6000]	NO STREAK
3	1,3000,[5000]	1 DAY, 1-1
4	6,5000,[8000 8000 1000 5000 8000 8000]	6 DAYS, 1-6
5	4,5000,[5000 5000 5000 5000]	4 DAYS, 1-4

Program 3: Squares of a Sorted Array (LeetCode 977, Medium, 20 min)

Problem Statement:

Given a sorted (ascending) array of integers (which may include negatives), return an array of the squares of each number, also sorted ascending.

Test Cases:

#	Input	Expected
1	5: [-4 -1 0 3 10]	0 1 9 16 100
2	5: [-7 -3 2 3 11]	4 9 9 49 121
3	3: [1 2 3]	1 4 9

#	Input	Expected
4	3: [-3 -2 -1]	1 4 9
5	1: [-5]	25

Program 4: Ride Sharing Load Balancer (Hard, 25 min)

Problem Statement:

N rides with start/end times. Fleet size F. Max concurrent vehicles needed? Sweep-line: +1 event at start, -1 at end, sort (tie-break -1 before +1 so touching rides share vehicle), scan for max concurrent.

Test Cases:

#	Input	Output
1	3 rides F=2, (0,30)(15,45)(60,90)	FLEET SUFFICIENT\nMAX CONCURRENT: 2
2	Same 3 rides F=1	MINIMUM FLEET REQUIRED: 2\nMAX CONCURRENT: 2
3	2 touching F=1 (0,30)(30,60)	FLEET SUFFICIENT\nMAX CONCURRENT: 1
4	3 nested F=2, (0,60)(10,50)(20,40)	MINIMUM FLEET REQUIRED: 3\nMAX CONCURRENT: 3
5	3 disjoint F=1	FLEET SUFFICIENT\nMAX CONCURRENT: 1

Program 5: Defuse the Bomb (LeetCode 1652, Hard, Stretch, 10 min)

Problem Statement:

Circular array `code[]` of length N and integer `k`.

- If k > 0: replace each code[i] with the sum of the next k elements (with wrap-around).
- If k < 0: replace each code[i] with the sum of the previous |k| elements (with wrap-around).
- If k == 0: replace each code[i] with 0.

Test Cases:

#	Input	Expected
1	code=[5 7 1 4] k=3	12 10 16 13
2	code=[1 2 3 4] k=0	0 0 0 0
3	code=[2 4 9 3] k=-2	12 5 6 13
4	code=[5] k=0	0
5	code=[1 2 3] k=2	5 4 3

Challenge 1: LC 1346 Check If N and Its Double Exist

Problem Statement:

Given an array `arr`, check whether there exist two indices i and j (i != j) such that `arr[i] == 2 * arr[j]`. Bootcamp approach: brute-force O(N^2).

Test Cases:

#	Input	Expected
1	4: [10 2 5 3]	true
2	4: [3 1 7 11]	false
3	7: [-2 0 10 -19 4 6 -8]	false
4	2: [0 0]	true
5	3: [-2 4 -4]	true

Challenge 2: LC 167 Two Sum II Sorted

Problem Statement:

Read N, target, and N sorted integers. Print the 1-indexed pair of positions (i j) whose values sum to target, or NO PAIR if none exists. *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	4 9 2 7 11 15	1 2
2	2 6 2 4	1 2
3	5 17 1 3 5 7 11	NO PAIR
4	5 6 -1 0 3 4 5	NO PAIR
5	6 11 1 2 3 4 5 6	5 6

Challenge 3: LC 643 Maximum Average Subarray I

Problem Statement:

Read N, K, and N integers. Print the maximum average over all contiguous K-element subarrays, formatted to 5 decimals. *(written by Claude as fallback; source had no Problem Statement)*

Test Cases:

#	Input	Expected Output
1	5 4 1 12 -5 -6 50	12.75000
2	5 1 1 2 3 4 5	5.00000
3	3 3 4 0 -2	0.66667
4	4 2 -1 -3 -5 -7	-2.00000

#	Input	Expected Output
5	2 2 1 2	1.50000

CDP 226 - String Processing Patterns

Program 1: Password Strength Checker (Easy, 15 min)

Problem Statement:

len<6 WEAK; len>=12 and all 4 classes VERY STRONG; len>=8 with digit+upper STRONG; else MEDIUM. Special = !@#$%.

Test Cases:

#	Input	Output
1	abc12	WEAK
2	simple77	MEDIUM
3	StrongP4ss	STRONG
4	My#SuperP4ssword	VERY STRONG
5	abc123	MEDIUM

Program 2: Student Name Anagram Matcher (Medium, 20 min)

Problem Statement:

Two names (fgets, may contain spaces). Case-insensitive, ignore spaces. Frequency-compare. Print ANAGRAMS or NOT ANAGRAMS.

Test Cases:

#	Input	Output
1	listen / silent	ANAGRAMS
2	hello / world	NOT ANAGRAMS
3	Elvis / Lives	ANAGRAMS
4	Astronomer / Moon starer	ANAGRAMS
5	Dormitory / Dirty Room	ANAGRAMS

Program 3: Substrings of Size Three with Distinct Characters (LeetCode 1876, Medium, 20 min)

Problem Statement:

Given a string `s` consisting of lowercase English letters, return the number of substrings of length 3 that contain three distinct characters.

Test Cases:

#	Input	Expected
1	xyzzaz	1
2	aababcabc	4
3	abc	1

#	Input	Expected
4	aaa	0
5	ab	0

Program 4: Address Parser and Validator (Hard, 30 min)

Problem Statement:

N addresses formatted `Name,DoorNo,Street,City,PIN`. Validate in order: Name (letters+spaces), DoorNo (starts with digit), Street (non-empty), City (letters+spaces), PIN (exactly 6 digits). Report first failure or VALID. Summary at end.

Test Cases:

#	Input	Output
1	2 valid (Alice, Eve)	2 VALID, 0 INVALID
2	4 each invalid in different field	DoorNo, DoorNo, PIN, City
3	1: Alice1,10,Main Rd,City,123456	INVALID: Name
4	1: Sam,1A,Street,City,100100	VALID
5	1: Sam,1,Street,City,12345A	INVALID: PIN

Program 5: Add Strings (LeetCode 415, Hard, Stretch, 10 min)

Problem Statement:

Given two non-negative integers `num1` and `num2` represented as strings, return their sum as a string. Do not use built-in big-integer libraries; implement digit-by-digit addition with carry.

Test Cases:

#	num1	num2	Expected
1	11	123	134
2	456	77	533
3	0	0	0
4	999	1	1000
5	1	999999	1000000

Challenge 1: LC 942 DI String Match

Problem Statement:

Given a string `s` of length N consisting only of 'I' (increasing) and 'D' (decreasing), construct a permutation `perm` of [0..N] such that:

- if s[i] == 'I' then perm[i] < perm[i+1]
- if s[i] == 'D' then perm[i] > perm[i+1]

Test Cases:

#	Input	Expected
1	IDID	0 4 1 3 2
2	III	0 1 2 3
3	DDI	3 2 0 1
4	I	0 1
5	D	1 0

Challenge 2: LC 541 Reverse String II

Problem Statement:

Given a string `s` and integer `k`, reverse the first k characters of every block of 2k characters. If fewer than k characters remain, reverse all of them. If between k and 2k characters remain, reverse the first k and leave the rest as-is.

Test Cases:

#	s	k	Expected
1	abcdefg	2	bacdfeg
2	abcd	2	bacd
3	a	2	a
4	abcdefg	8	gfedcba
5	abcde	3	cbade

Challenge 3: LC 1456 Maximum Number of Vowels in a Substring of Given Length

Problem Statement:

Given a string `s` (lowercase only) and integer `k`, return the maximum number of vowels in any contiguous substring of length k. Vowels: a, e, i, o, u.

Test Cases:

#	s	k	Expected
1	abciiidef	3	3
2	aeiou	2	2
3	leetcode	3	2
4	rhythms	4	0
5	tryhard	4	1

CDP 227 - Data Structure Intuition

Program 1: Undo Feature in Text Editor (Easy, 10 min)

Problem Statement:

A simple text editor supports three operations, one per line:

- `TYPE <text>`: push the text onto the undo stack.
- `UNDO`: pop the top entry (if any). If stack is empty, do nothing.
- `SHOW`: print the current top of stack, or `EMPTY` if stack is empty.
- `END`: stop processing input.

Read operations until `END` is encountered.

Test Cases:

#	Input operations	Output
1	TYPE hello / SHOW / TYPE world / SHOW / UNDO / SHOW / END	hello / world / hello
2	UNDO / SHOW / TYPE a / SHOW / END	EMPTY / a
3	TYPE first / TYPE second / END	(no output)
4	TYPE x / UNDO / SHOW / END	EMPTY
5	TYPE a / TYPE b / TYPE c / UNDO / SHOW / UNDO / SHOW / END	b / a

Program 2: Duplicate Detector in Customer Database (Medium, 20 min)

Problem Statement:

A customer database has N phone number entries (each an 11-character string of digits; duplicates possible). Find all phone numbers that appear more than once, along with their occurrence counts. Print output in ASCENDING alphabetical order of phone number strings.

Print header "DUPLICATES:" first. Then one line per duplicated number: "<phone>: <count> OCCURRENCES". After all duplicates, print "TOTAL DUPLICATE GROUPS: <count>". If no duplicates exist, print "NO DUPLICATES FOUND" after the header.

Test Cases:

#	Input	Expected Output
1	5 / 9876543210 8765432109 9876543210 7654321098 8765432109	DUPLICATES / 8765432109 (2) / 9876543210 (2) / TOTAL 2
2	3 / 9876543210 8765432109 7654321098	DUPLICATES / NO DUPLICATES FOUND
3	1 / 9999999999	DUPLICATES / NO DUPLICATES FOUND
4	4 / 1111111111 x4	DUPLICATES / 1111111111 (4) / TOTAL 1
5	6 / aaa bbb aaa ccc bbb ddd	DUPLICATES / aaa (2) / bbb (2) / TOTAL 2

Program 3: Baseball Game (LeetCode 682, Medium, 20 min)

Problem Statement:

You are keeping the score for a baseball game. You're given a list of strings `ops`, where each `ops[i]` is one of these operations:

- An integer x: record a new score x.
- "+": record a new score that is the sum of the previous two scores. Both previous scores remain valid.
- "D": record a new score that is double the previous score. The previous score remains valid.
- "C": invalidate the previous score, removing it from the record. The previous score remains as the new "previous score".

Return the sum of all scores after applying all operations.

Test Cases:

#	Operations	Expected Output
1	5 2 C D +	30
2	5 -2 4 C D 9 + +	27
3	1	1
4	1 2 +	6
5	3 D C 5	8

Program 4: Hospital Token Management System (Hard, 25 min)

Problem Statement:

A hospital has two queues: emergency (E) and general (G). Emergency patients are served before general patients. Within each queue, FIFO order applies.

Operations, one per line:

- `ARRIVE E <id>`: add patient <id> to emergency queue.
- `ARRIVE G <id>`: add patient <id> to general queue.
- `CALL`: call the next patient. Print "CALLING <id>". If both queues empty, print "QUEUE EMPTY".
- `STATUS`: print "E: x, G: y" where x and y are the current sizes of the two queues.
- `END`: stop processing.

Test Cases:

#	Input	Output
1	ARRIVE E 101, G 202, E 103, then 4 CALLs	CALLING 101, 103, 202, QUEUE EMPTY
2	ARRIVE E 1, G 2, G 3, STATUS, CALL, STATUS	E:1 G:2, CALLING 1, E:0 G:2

#	Input	Output
3	CALL, STATUS	QUEUE EMPTY, E:0 G:0
4	ARRIVE G 50, G 51, 2 CALLs	CALLING 50, 51
5	ARRIVE G 10, E 20, G 11, 3 CALLs	CALLING 20, 10, 11

Program 5: Final Prices With a Special Discount in a Shop (LeetCode 1475, Hard, Stretch, 15 min)

Problem Statement:

A shop has prices for N items. The discount applied to item i is: the next price prices[j] (for some j > i) that is less than or equal to prices[i]. If no such j exists, no discount applies.

Return an array `final[i] = prices[i] - discount[i]` for each item.

Test Cases:

#	Input (N, prices)	Expected Output
1	5 / 8 4 6 2 3	4 2 4 2 3
2	5 / 1 2 3 4 5	1 2 3 4 5
3	4 / 10 1 1 6	9 0 1 6
4	1 / 5	5
5	4 / 10 5 2 1	5 3 1 1

Challenge 1: Count the Number of Consistent Strings (LeetCode 1684, Easy)

Problem Statement:

You are given an `allowed` string of distinct lowercase letters and N words. A word is "consistent" if every character in the word appears in the `allowed` string. Return the count of consistent words.

Test Cases:

#	allowed	words	Expected Output
1	ab	ad, bd, aaab, baa, badab	2
2	abc	a, b, c, ab, ac, bc, abc	7
3	cad	cc, acd, b, ba, bac, bad, ac, d	4
4	z	z, zz, zzz	3
5	a	b, c, d	0

Challenge 2: Maximum Nesting Depth of the Parentheses (LeetCode 1614, Easy)

Problem Statement:

Given a valid parenthesis string `s` (containing operators, digits, and parentheses, with all parentheses balanced), return the maximum nesting depth of the parentheses.

The depth of `()` is 1, of `(())` is 2, of `((()))` is 3, etc.

Test Cases:

#	Input	Expected Output
1	((1+(2*3))+((1)/2))	3
2	(1+(2*3)+((8)/4))+1	3
3	1+(2*3)/(2-1)	1
4	1	0
5	(((((1)))))	5

Challenge 3: Uncommon Words from Two Sentences (LeetCode 884, Easy)

Problem Statement:

A sentence is a string of single-space-separated words. A word is "uncommon" if it appears exactly once in one sentence AND does not appear in the other sentence.

Given two sentences `s1` and `s2`, return all uncommon words. Output in alphabetical order, space-separated. If there are no uncommon words, print "NONE".

Test Cases:

#	s1	s2	Expected Output
1	this apple is sweet	this apple is sour	sour sweet
2	apple apple	banana	banana
3	a b c	a c b	NONE
4	fizz buzz	fizz bazz	bazz buzz
5	the quick	the lazy	lazy quick

CDP 228 - Search and Sort Foundations

Program 1: Phone Directory Search (Easy, 15 min)

Problem Statement:

A phone directory has N entries, each with a name and phone number, sorted by name in ascending order. Given Q query names, for each query print:

- "FOUND: <name> <phone>" if the name exists.
- "NOT FOUND: <query>" if it does not.

Each query must use O(log N) time (binary search, not linear scan).

Test Cases:

#	Setup	Query	Expected Output
1	4 entries	Carol	FOUND: Carol 7654321098
2	3 entries (Alice, Bob, Carol)	Zara	NOT FOUND: Zara
3	3 entries	Alice (first)	FOUND: Alice 1
4	3 entries	Carol (last)	FOUND: Carol 3
5	3 entries, 3 queries	Bob / Zara / Alice	3 lines respectively

Program 2: Binary Search (LeetCode 704, Medium, 15 min)

Problem Statement:

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, return its 0-based index. Otherwise, return -1. The algorithm must run in O(log n) time.

Test Cases:

#	Input (N, target, nums)	Expected Output
1	6, 9, [-1 0 3 5 9 12]	4
2	6, 2, [-1 0 3 5 9 12]	-1
3	1, 5, [5]	0
4	1, 5, [7]	-1
5	5, 1, [1 3 5 7 9]	0

Program 3: Bus Seat Availability Finder (Medium, 20 min)

Problem Statement:

A bus has S numbered seats (1 to S). A sorted list of B BOOKED seat numbers is given (in ascending order). Find the FIRST (smallest) seat number that is not booked, using binary search. Print "FIRST AVAILABLE: X". If all S seats are booked, print "FULL".

Test Cases:

#	Input (S, B, booked)	Expected Output
1	10, 4, [1 2 4 5]	FIRST AVAILABLE: 3
2	10, 4, [1 2 3 4]	FIRST AVAILABLE: 5
3	5, 5, [1 2 3 4 5]	FULL
4	10, 0, []	FIRST AVAILABLE: 1
5	10, 3, [2 3 4]	FIRST AVAILABLE: 1

Program 4: Event Scheduler Conflict-Free (Hard, 30 min)

Problem Statement:

A campus has N events to schedule in a single venue, each with a name, start time, and end time. Only one event can run at a time in the venue. Events can be scheduled back-to-back (an event ending at time t and another starting at time t is fine).

Find the MAXIMUM number of events that can be scheduled without conflict. Print the count and the chosen events in the order they will be held.

Test Cases:

#	Input	Expected Output
1	3: A 1-3, B 3-5, C 5-7	MAX 3 / A, B, C
2	4: A 1-4, B 2-3, C 3-5, D 5-7	MAX 3 / B, C, D
3	1: A 0-10	MAX 1 / A
4	3 all overlap: A 1-10, B 2-9, C 3-8	MAX 1 / C (earliest end)
5	4: A 5-8, B 1-2, C 3-4, D 0-1	MAX 4 / D, B, C, A

Program 5: Third Maximum Number (LeetCode 414, Hard, Stretch, 15 min)

Problem Statement:

Given an integer array `nums`, return the third DISTINCT maximum number in this array. If the third distinct maximum does not exist, return the maximum number.

Test Cases:

#	Input (N, nums)	Expected Output
1	3, [3 2 1]	1
2	2, [1 2]	2
3	4, [2 2 3 1]	1
4	3, [5 5 5]	5
5	6, [1 2 2 5 3 5]	2

Challenge 1: Count Negative Numbers in a Sorted Matrix (LeetCode 1351, Easy)

Problem Statement:

Given an `m x n` grid where each row is sorted in non-increasing order and each column is also sorted in non-increasing order, return the number of negative numbers in the grid.

Test Cases:

#	Input (M, N, grid)	Expected Output
1	4, 4, [[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]]	8
2	2, 2, [[3,2],[1,0]]	0
3	1, 1, [[-1]]	1
4	1, 3, [[5,1,0]]	0
5	1, 3, [[-1,-1,-1]]	3

Challenge 2: Sqrt(x) (LeetCode 69, Medium)

Problem Statement:

Given a non-negative integer x, compute and return the square root of x. Since the return type is an integer, the decimal digits are truncated, and only the integer part of the result is returned. Do NOT use built-in functions like `sqrt` or `pow`.

Test Cases:

#	Input (x)	Expected Output
1	4	2
2	8	2
3	0	0
4	1	1
5	2147395599	46339

Challenge 3: Peak Index in a Mountain Array (LeetCode 852, Medium)

Problem Statement:

A mountain array is an array that strictly increases up to some index (the peak) and then strictly decreases. Given such an array, find the peak index in O(log N).

Test Cases:

#	Input	Expected Output
1	3: [0 1 0]	1
2	4: [0 2 1 0]	1
3	6: [0 10 5 2 1 0]	1
4	3: [0 5 4]	1

#	Input	Expected Output
5	7: [1 2 3 4 3 2 1]	3

CDP 229 - Company Simulation Block 1 (Accenture-Style)

Problem 1: Attendance Summary Report (Medium, 30 min)

Problem Statement:

A college engineering department is generating the semester-end attendance summary. For each student, you are given:

- name (a single word, no spaces, up to 31 characters)
- classes_attended (non-negative integer)
- total_classes (positive integer)

Compute each student's attendance percentage and classify:

- `ELIGIBLE` if percentage >= 75.00
- `NOT-ELIGIBLE` if percentage < 75.00

For each student, print on a single line: `NAME PERCENTAGE STATUS` with percentage rounded to 2 decimals.

After all students, print the summary (exact format): `TOTAL STUDENTS: <n> ELIGIBLE: <e> NOT-ELIGIBLE: <ne> HIGHEST ATTENDANCE: <name> <pct> LOWEST ATTENDANCE: <name> <pct>`

If multiple students tie for highest or lowest, report the EARLIEST (smallest index) in the input.

If N = 0, print `NO STUDENTS` and stop.

Test Cases:

#	Input	Expected Output Summary
1	5: Alice 40/50, Bob 32/50, Carol 30/50, David 38/50, Eve 45/50	Alice 80.00 ELIGIBLE, Bob 64.00 NOT-ELIGIBLE, Carol 60.00 NOT-ELIGIBLE, David 76.00 ELIGIBLE, Eve 90.00 ELIGIBLE / TOTAL 5 / ELIGIBLE 3 / NOT-ELIGIBLE 2 / HIGHEST Eve 90.00 / LOWEST Carol 60.00
2	3 all eligible: A 45/50, B 40/50, C 50/50	A 90.00, B 80.00, C 100.00 all ELIGIBLE / TOTAL 3 / ELIGIBLE 3 / NOT-ELIGIBLE 0 / HIGHEST C 100.00 / LOWEST B 80.00
3	2 at boundary: A 75/100, B 74/100	A 75.00 ELIGIBLE, B 74.00 NOT-ELIGIBLE / TOTAL 2 / ELIGIBLE 1 / NOT-ELIGIBLE 1 / HIGHEST A / LOWEST B
4	1: Alice 90/100	Alice 90.00 ELIGIBLE / TOTAL 1 / ELIGIBLE 1 / NOT-ELIGIBLE 0 / HIGHEST Alice / LOWEST Alice
5	3 all not-eligible: A 10/50, B 20/50, C 30/50	20%, 40%, 60% all NOT-ELIGIBLE / TOTAL 3 / ELIGIBLE 0 / NOT-ELIGIBLE 3 / HIGHEST C 60.00 / LOWEST A 20.00

Problem 2: Salary Slip Generator (Medium, 30 min)

Problem Statement:

A payroll system generates monthly salary slips. For each of N employees, you are given:

- name (single word, no spaces, up to 31 characters)
- basic (basic salary in rupees per month, positive floating-point or integer)

Compute using these rules (all percentages on BASIC unless specified):

1. HRA = 20% of basic.
2. DA = 10% of basic.
3. GROSS = basic + HRA + DA.
4. PF deduction = 12% of basic.
5. Annual gross = GROSS * 12.
6. Annual tax (slab-based, applied on ANNUAL gross):

- Up to Rs 5,00,000: no tax.
- Rs 5,00,001 to Rs 10,00,000: 10% on the portion above 5L.
- Above Rs 10,00,000: flat Rs 50,000 on the 5L-10L slab + 20% on the portion above 10L.

1. Monthly tax = annual_tax / 12.
2. NET = GROSS - PF - monthly_tax.

For each employee, print on a single line: `NAME BASIC HRA DA GROSS PF TAX NET` (all amounts with 2 decimals, space-separated).

After all employees, print: `TOTAL PAYROLL NET: <sum>` (2 decimals).

Test Cases:

#	Input	Notes
1	1: Alice 30000	Annual 468000 below 5L, no tax. Net = 35400.00
2	1: Bob 60000	Annual 936000 in 5-10L. Tax on (936000-500000)*0.10 = 43600. Monthly 3633.33. Net 67166.67
3	1: Carol 100000	Annual 1560000. Tax = 50000 + (1560000-1000000)*0.20 = 162000. Monthly 13500. Net 104500.00
4	3 mixed: A 25000, B 50000, C 80000	Individual nets: 29500, 56666.67, 86100. Total 172266.67
5	1: Sam 20000	Annual 312000 below 5L. Net = 23600.00

Problem 3: Library Fine Calculator (Medium, 30 min)

Problem Statement:

A library fine system computes overdue fines for returned books. For N books, each given as:

- due_date (integer, days from reference date 0)

- return_date (integer, days from reference date 0)

Overdue = return_date - due_date. If overdue <= 0, no fine (book was on time or early).

Fine is computed slab-by-slab on the overdue days:

- Days 1-7: Rs 2 per day.
- Days 8-30: Rs 5 per day (23-day span).
- Days 31+: Rs 10 per day.

For each book (1-indexed):

- If not overdue: `BOOK <n>: ON TIME, NO FINE`
- If overdue: `BOOK <n>: OVERDUE <d> DAYS, FINE <amount>`

At the end, print: `TOTAL FINE: <sum> BOOKS OVERDUE: <count>`

Test Cases:

#	Input	Expected Output
1	3 on-time: (10,8),(20,20),(30,25)	All "ON TIME, NO FINE" / TOTAL FINE 0 / BOOKS OVERDUE 0
2	1 slab-1 only: due 10, return 15 (5 days)	OVERDUE 5, FINE 10
3	1 slab-1+2: due 10, return 30 (20 days)	FINE 14 + 65 = 79
4	1 all 3 slabs: due 10, return 50 (40 days)	FINE 14 + 115 + 100 = 229
5	4 mix: (10,5), (20,25), (30,40), (50,100)	ON TIME, FINE 10, FINE 29, FINE 329. Total 368. Overdue count 3.

CDP 230 - Company Simulation Block 2 (Visa-Style)

Problem 1: Sliding Window Maximum Engagement (Medium-Hard, 30 min)

Problem Statement:

A video streaming platform records user engagement score per minute over N minutes. Given the engagement scores and a window size K (in minutes), find the K-minute window with the HIGHEST AVERAGE engagement and report:

- The average (rounded to 2 decimals).
- The start minute (1-indexed) of that window.
- The end minute (1-indexed).

If multiple windows tie for the highest average, report the EARLIEST start.

Constraints:

- 1 <= K <= N <= 100,000
- Engagement scores fit in int (can be negative).
- Algorithm must run in O(N) time.

If N <= 0 or K <= 0 or K > N, print `INVALID WINDOW` and stop.

Test Cases:

#	Input (N, K, scores)	Expected Output
1	5, 4, [1 12 -5 -6 50]	MAX AVG 12.75, START 2, END 5
2	4, 1, [10 5 20 15]	MAX AVG 20.00, START 3, END 3
3	3, 3, [4 5 6]	MAX AVG 5.00, START 1, END 3
4	3, 5, [1 2 3]	INVALID WINDOW
5	5, 3, [3 3 3 3 3]	MAX AVG 3.00, START 1, END 3

Problem 2: Inventory Duplicate Auditor (Medium-Hard, 30 min)

Problem Statement:

A warehouse inventory has N SKU scans (integer product IDs, non-negative, possibly duplicated due to scanning errors). Task:

1. Find all SKUs that appear 2 or more times. Print them in ASCENDING order of SKU with their occurrence counts, one per line, in the format `SKU COUNT`.
2. Identify the SKU with the HIGHEST occurrence count. On ties, prefer the SMALLEST SKU value.
3. Report the total count of DISTINCT SKUs.

Output format: `DUPLICATES (count >= 2): <sku> <count> <sku> <count> ... MOST DUPLICATED: <sku> (<count>) DISTINCT SKUS: <n>`

If no duplicates exist, the duplicates section shows a single line `NONE` and the MOST DUPLICATED line shows `MOST DUPLICATED: NONE`.

If N = 0, print the header followed by NONE, NONE, and DISTINCT SKUS: 0.

Test Cases:

#	Input	Expected Output
1	7: 100 200 100 300 200 100 400	100(3), 200(2) / MOST 100(3) / DISTINCT 4
2	5: 1 2 3 4 5	NONE / MOST NONE / DISTINCT 5
3	4: 7 7 7 7	7(4) / MOST 7(4) / DISTINCT 1
4	1: 42	NONE / MOST NONE / DISTINCT 1
5	6: 10 20 10 20 30 40	10(2), 20(2) / MOST 10(2) / DISTINCT 4

Problem 3: Campus Event Popularity Ranker (Hard, 30 min)

Problem Statement:

A campus has N events. Each event has:

- name (single word, no spaces, up to 31 characters)
- capacity C (positive integer, max attendees)
- signups S (non-negative integer, can exceed capacity when oversubscribed)

Compute for each event:

- Popularity SCORE = min(S, C). (Oversubscribed events only count capacity.)
- FILL percentage = SCORE * 100 / C (rounded to 2 decimals).

Print all events sorted by:

1. SCORE descending (higher popularity first).
2. FILL percentage descending (on score ties, fuller event first).
3. Name ascending alphabetical (on both ties, alphabetical first).

Format per line: `<rank>. <name> SCORE=<score> FILL=<pct>%.`

After the ranked list, print: `TOP EVENT: <name> FULL EVENTS: <count where score == capacity>`

If N = 0, print `NO EVENTS` and stop.

Test Cases:

#	Input	Expected Output (key lines)
1	4: Hackathon(100,150), Workshop(50,30), Talk(80,80), Coding(200,50)	Rank 1 Hackathon (100, 100%), 2 Talk (80, 100%), 3 Coding (50, 25%), 4 Workshop (30, 60%). TOP Hackathon, FULL 2.
2	3 all full: A(50,60), B(40,40), C(30,30)	Rank 1 A (50, 100%), 2 B (40, 100%), 3 C (30, 100%). TOP A, FULL 3.
3	3 tied score, different fill: Alpha(100,50), Beta(50,50), Gamma(200,50)	Rank 1 Beta (50, 100%), 2 Alpha (50, 50%), 3 Gamma (50, 25%). TOP Beta. FULL 1.
4	1: Solo(100,80)	Rank 1 Solo (80, 80%). TOP Solo, FULL 0.

#	Input	Expected Output (key lines)
5	0 events	NO EVENTS

CDP 231 - Company Simulation Block 3 (Mixed-Pool)

Problem 1: Multi-Category Discount Voucher System (Medium, 30 min)

Problem Statement:

An e-commerce checkout applies discounts to an N-item order. Each line item has:

- name (single word, up to 31 chars)
- category (one of: ELECTRONICS, CLOTHING, GROCERY, BOOKS)
- price (positive integer, in rupees)

Per-item discount rules:

- ELECTRONICS: 10% off if price > 5000, else 0%.
- CLOTHING: 15% off if price > 1000, else 5%.
- GROCERY: 5% off if price > 500, else 0%.
- BOOKS: 20% off always (any price).

Compute the after-discount price of each item. Sum into subtotal. Apply cart-level discount:

- If subtotal > 10000: additional 5% off subtotal.
- Else: no cart discount.

Output (per line item): `<n> <category> <price> <discount_pct>% <after_discount_price>` (discount_pct printed as integer with %, after_discount_price with 2 decimals).

After all items: `SUBTOTAL: <subtotal> CART DISCOUNT: <cart_disc_amount> FINAL: <final>`

If N = 0, print `NO ITEMS`.

Test Cases:

#	Input (summary)	Output (key lines)
1	4: Laptop ELECTRONICS 60000, Tshirt CLOTHING 500, Bread GROCERY 100, Novel BOOKS 500	Laptop 10%, Tshirt 5%, Bread 0%, Novel 20%. SUBTOTAL 54975, CART DISC 2748.75, FINAL 52226.25
2	4: TV ELECTRONICS 8000, Shirt CLOTHING 2000, Oil GROCERY 800, Mag BOOKS 200	All in-threshold. SUBTOTAL 9820, CART DISC 0, FINAL 9820
3	2 books: Book1 100, Book2 200	SUBTOTAL 240, CART DISC 0, FINAL 240
4	1: Phone ELECTRONICS 30000	Phone 10% off. SUBTOTAL 27000, CART DISC 1350, FINAL 25650
5	1: Accessory ELECTRONICS 3000 (below threshold)	0% disc. SUBTOTAL 3000, CART DISC 0, FINAL 3000

Problem 2: Clustered Task Assignment (Hard, 30 min)

Problem Statement:

A job queue has N tasks to assign to M workers. Each task has:

- name (single word, up to 31 chars)
- duration (positive integer, in minutes)
- deadline (positive integer, in minutes from time 0)

Every worker starts at time 0 and can handle one task at a time (sequentially).

Scheduling rule:

1. Sort tasks by deadline ASCENDING. Tie-breaks: shorter duration first, then alphabetical name.
2. Process tasks in this order. Assign each task to the worker who will finish EARLIEST (smallest `worker_end` time). Ties on worker end time: smaller worker index wins.
3. The task's START = worker_end before assignment. END = START + duration.
4. After assignment, worker_end is updated to END.
5. If END <= deadline, the task is ON-TIME. Otherwise, LATE. Either way, the task IS assigned; lateness is just tracked.

Output per task (in the order tasks are processed): `TASK <n>: WORKER <w>, START <s>, END <e>, <STATUS>` (worker is 1-indexed).

After all tasks: `ON-TIME: <c> LATE: <c> MAKESPAN: <max worker_end across all workers>`

If N = 0 or M = 0: print `NO ASSIGNMENT`.

Test Cases:

#	Input	Key Output
1	3 tasks 2 workers: T1(5,10), T2(3,8), T3(2,20)	T2 first (earliest deadline), then T1, then T3. All ON-TIME, MAKESPAN 5
2	3 tasks 1 worker: T1(5,5), T2(3,10), T3(2,20)	All ON-TIME, MAKESPAN 10
3	2 tasks 5 workers: T1(10,15), T2(10,15)	Both ON-TIME, MAKESPAN 10
4	3 tasks 1 worker, tight: T1(10,5), T2(5,12), T3(5,30)	T1 LATE (ends 10, deadline 5). T2 LATE (ends 15, deadline 12). T3 ON-TIME
5	3 tasks 2 workers, reverse-order input: C(5,30), B(5,20), A(5,10)	Sort by deadline: A, B, C. Assign all. MAKESPAN 10

Problem 3: Subscription Revenue Analytics (Hard, 30 min)

Problem Statement:

A SaaS platform has N months of subscription revenue (non-negative 64-bit integers; long long in C). Given a rolling window size K in months, compute for each valid window:

- Rolling K-month revenue sum.

Also identify:

- BEST K-month window (maximum rolling sum).
- WORST K-month window (minimum rolling sum).
- PEAK MONTH (single month with highest revenue; earliest wins on ties).

On ties for best/worst window, prefer EARLIEST start month.

Output: `ROLLING SUMS: WINDOW <start>-<end>: <sum> WINDOW <start>-<end>: <sum> ... (for each valid window in order) BEST WINDOW: <start>-<end> (<sum>) WORST WINDOW: <start>-<end> (<sum>) PEAK MONTH: <m> (<revenue>)`

Months are 1-indexed.

If N <= 0, K <= 0, or K > N: print `INVALID`.

Test Cases:

#	Input	Key Output
1	N=6, K=3: [100, 200, 300, 50, 400, 500]	Windows 600, 550, 750, 950. BEST 4-6 (950). WORST 2-4 (550). PEAK 6 (500).
2	N=5, K=1: [10, 5, 20, 15, 8]	Windows 10,5,20,15,8. BEST 3-3 (20). WORST 2-2 (5). PEAK 3 (20).
3	N=4, K=4: [1, 2, 3, 4]	Single window 10. BEST=WORST=1-4 (10). PEAK 4 (4).
4	N=3, K=5: INVALID	INVALID
5	N=5, K=2 all 10s	All windows 20. BEST=WORST=1-2 (earliest). PEAK 1.

CDP 232 - Pattern Synthesis and Placement Playbook

Program 1: Most Active Reviewer (Easy, 10 min)

Problem Statement:

A product review platform receives N review events. Each event records a reviewer ID (integer 1-100). Compute:

- Total reviews per reviewer (only for reviewers with at least 1 review).
- MOST ACTIVE reviewer (highest review count). On ties, smallest ID wins.
- LEAST ACTIVE reviewer (smallest review count among those with >=1 review). On ties, smallest ID wins.

Output format: `REVIEW COUNTS: REVIEWER <id>: <count> ... (for each reviewer with >=1 review, ascending ID order) MOST ACTIVE: REVIEWER <id> (<count>) LEAST ACTIVE: REVIEWER <id> (<count>)`

If N = 0, print `NO REVIEWS`.

Test Cases:

#	Input	Key Output
1	8: [1 2 1 3 1 2 5 1]	Counts 1:4, 2:2, 3:1, 5:1. Most 1(4). Least 3(1).
2	3: [7 7 7]	Count 7:3. Most 7(3). Least 7(3).
3	0	NO REVIEWS
4	1: [42]	Count 42:1. Most 42(1). Least 42(1).
5	4: [1 2 1 2] (tie on count)	Most 1(2). Least 1(2). (Tie: smallest ID both ways.)

Program 2: Smart Thermostat Controller (Medium, 20 min)

Problem Statement:

A smart thermostat monitors temperature and adjusts mode each hour. Given N hourly temperature readings (integer Celsius), determine:

1. Per-hour mode based on temperature:

- temperature < 18°C → `BOOST` (5 units per hour)
- 18°C <= temperature <= 26°C → `COMFORT` (2 units per hour)
- temperature > 26°C → `ECO` (3 units per hour)

1. Total energy consumed = sum of per-hour units.
2. Mode transitions = count of times the mode CHANGES between consecutive hours (hour 1 does not count; only hours 2..N count toward transitions).
3. Dominant mode = mode used for the most hours. On ties, use the priority order BOOST < COMFORT < ECO (so BOOST wins over COMFORT, which wins over ECO).

Output per hour: `HOUR <n>: <temp>C <MODE> <units>`

After all hours: `TOTAL ENERGY: <sum> MODE TRANSITIONS: <count> DOMINANT MODE: <mode>`

If N = 0, print `NO DATA`.

Test Cases:

#	Input	Key Output
1	6: [15 22 28 20 16 30]	3 modes each 2 hours. Total 20. Transitions 5. Dominant BOOST (priority tie-break).
2	4: [22 22 22 22]	All COMFORT. Total 8. Transitions 0. Dominant COMFORT.
3	4: [18 26 17 27]	C-C-B-E. Total 12. Transitions 2 (C→B, B→E). Dominant COMFORT.
4	1: [20]	COMFORT only. Total 2. Transitions 0. Dominant COMFORT.
5	0	NO DATA

Program 3: Corporate Flight Bookings (LeetCode 1109, Medium, 25 min)

Problem Statement:

There are N flights labeled 1 to N. You are given an array of bookings where `bookings[i] = [first_i, last_i, seats_i]`. This means `seats_i` seats were booked on every flight from `first_i` to `last_i` inclusive (1-indexed).

Return an array `answer` of length N, where `answer[j]` is the total number of seats booked on flight j+1.

Test Cases:

#	Input (N, M, bookings)	Expected Output
1	5, 3, [[1,2,10],[2,3,20],[2,5,25]]	10 55 45 25 25
2	2, 2, [[1,2,10],[2,2,15]]	10 25
3	3, 0, []	0 0 0
4	1, 1, [[1,1,5]]	5
5	4, 1, [[1,4,100]]	100 100 100 100

Program 4: Top-Selling Products Dashboard (Hard, 25 min, CAPSTONE)

Problem Statement:

An e-commerce platform logs N sales events over a day. Each event records a product_id (1..M) and a quantity (positive integer). Task:

1. Compute the TOTAL QUANTITY sold per product.
2. Rank products by total quantity DESCENDING. On ties, smaller product ID first.
3. Print TOP K PRODUCTS (by rank).

4. Find the K-CONTIGUOUS window in the sorted-descending list whose combined quantity is highest. On ties, earliest window start.

Input: first line is `N M K` (number of events, max product ID, window size). Next N lines are `product_id quantity`.

Output: `TOTAL SALES: <total quantity across all events> TOP K PRODUCTS: <rank>. PRODUCT <id> QTY <q> ... (K lines) K-WINDOW PEAK: RANKS <start>-<end> QTY <peak_qty>`

Edge cases:

- If N = 0 or no products have any sales: print `NO SALES`.
- If K > number of distinct products sold: clamp K to that number.
- If K < 1: clamp K to 1.

Test Cases:

#	Input	Expected Output
1	7 5 3: (1,10)(2,5)(1,3)(3,8)(2,7)(4,15)(5,2)	TOTAL 50. Top: 4(15), 1(13), 2(12). Peak: ranks 1-3 qty 40.
2	5 3 1: (1,10)(2,20)(3,5)(1,5)(2,10)	TOTAL 50. Top: 2(30). Peak ranks 1-1 qty 30.
3	3 10 5 (K clamped): (1,5)(3,10)(1,5)	TOTAL 20. Top: 1(10), 3(10). Peak ranks 1-2 qty 20.
4	0 events	NO SALES
5	4 3 3: (1,5)(2,10)(3,3)(1,2)	TOTAL 20. Top: 2(10), 1(7), 3(3). Peak ranks 1-3 qty 20.

Program 5: Max Consecutive Ones III (LeetCode 1004, Hard, Stretch, 15 min)

Problem Statement:

Given a binary array `nums` and an integer `k`, return the maximum number of consecutive 1s in the array if you can flip at most `k` 0s.

Equivalent rephrasing: find the longest contiguous subarray containing at most `k` zeros.

Test Cases:

#	Input (N, k, nums)	Expected Output
1	11, 2, [1 1 1 0 0 0 1 1 1 1 0]	6
2	7, 0, [0 0 1 1 1 0 0]	3
3	4, 2, [1 1 1 1]	4
4	4, 2, [0 0 0 0]	2
5	5, 1, [1 0 1 0 1]	3

Challenge 1: Array Partition (LeetCode 561, Medium)

Problem Statement:

Given an integer array `nums` of 2N integers, group these integers into N pairs (a1, b1), (a2, b2), ..., (aN, bN) such that the sum of `min(ai, bi)` for all i is MAXIMIZED. Return the maximum sum.

Test Cases:

#	Input	Expected Output
1	4: [1 4 3 2]	4
2	6: [6 2 6 5 1 2]	9
3	2: [1 2]	1
4	2: [-1 -2]	-2
5	4: [3 3 3 3]	6

Challenge 2: Find All Anagrams in a String (LeetCode 438, Medium)

Problem Statement:

Given two strings `s` and `p`, return all start indices of p's anagrams in s. You may return the answer in any order (this problem prints them in ascending order).

Strings contain only lowercase English letters. An anagram is a permutation of p.

Test Cases:

#	Input (s, p)	Expected Output
1	cbaebabacd / abc	0 6
2	abab / ab	0 1 2
3	abcdef / zyx	NONE
4	abc / abcdef (p longer)	NONE
5	abc / abc	0

Challenge 3: Flipping an Image (LeetCode 832, Medium)

Problem Statement:

Given an `m x n` binary matrix (each entry 0 or 1), flip it HORIZONTALLY and then INVERT each value.

- Horizontal flip: reverse each row. For example, [1, 1, 0] becomes [0, 1, 1].
- Invert: replace every 0 with 1 and every 1 with 0. For example, [0, 1, 1] becomes [1, 0, 0].

Return the resulting matrix.

Test Cases:

#	Input (M, N, grid)	Expected Output
1	3, 3, [[1,1,0],[1,0,1],[0,0,0]]	[[1,0,0],[0,1,0],[1,1,1]]
2	4, 4, [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]	[[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
3	1, 1, [[1]]	[[0]]

#	Input (M, N, grid)	Expected Output
4	1, 2, [[0,1]]	[[0,1]]
5	2, 3, [[1,1,1],[0,0,0]]	[[0,0,0],[1,1,1]]